Fine-Tuning Large Language Models

Vinay Setty
vinay.j.setty@uis.no

University of Stavanger

Information Access & Artificial Intelligence

Department of Electrical Engineering and Computer Science
University of Stavanger

February 20, 2026

Fine-Tuning vs Frozen Models

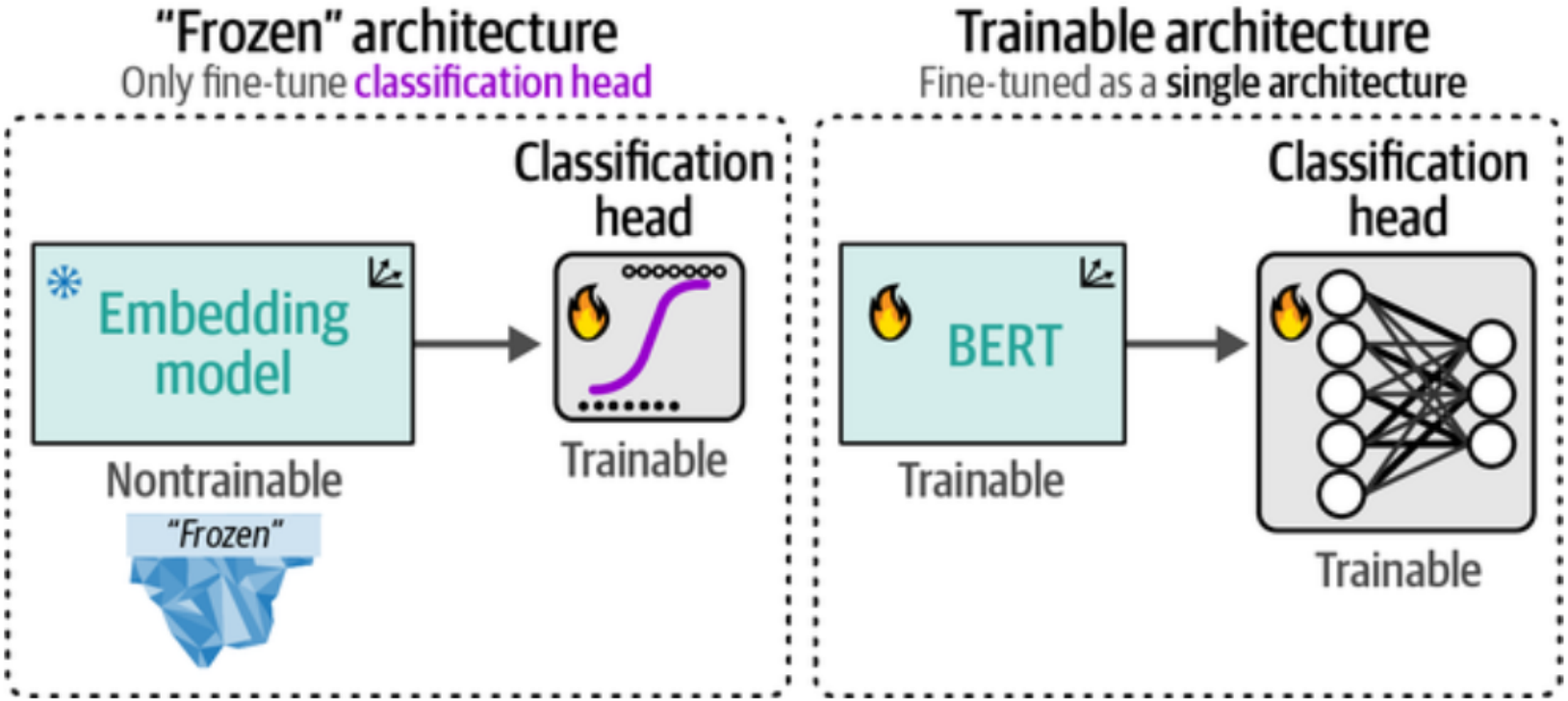
Frozen model

- Pretrained encoder used as feature extractor
- Only classification head is trainable
- Fast but limited task adaptation

Full fine-tuning

- Update encoder + classification head
- Task-specific representations learned
- Higher performance, higher compute

Fine-Tuning vs Frozen Models (cont.)



Fine-Tuning BERT for Classification

Model:

- Pretrained encoder $f_\theta(x)$
- Classification head g_ϕ

Prediction:

$$\hat{y} = \text{softmax}(g_\phi(f_\theta(x)))$$

Loss:

$$\mathcal{L} = -\sum y \log \hat{y}$$

Training updates:

$$\theta, \phi \leftarrow \theta, \phi - \eta \nabla \mathcal{L}$$

Observation: Full fine-tuning improves F1 compared to frozen encoder.

Freezing Layers

Strategy:

- Freeze embeddings
- Freeze first k encoder blocks
- Train final layers + classifier

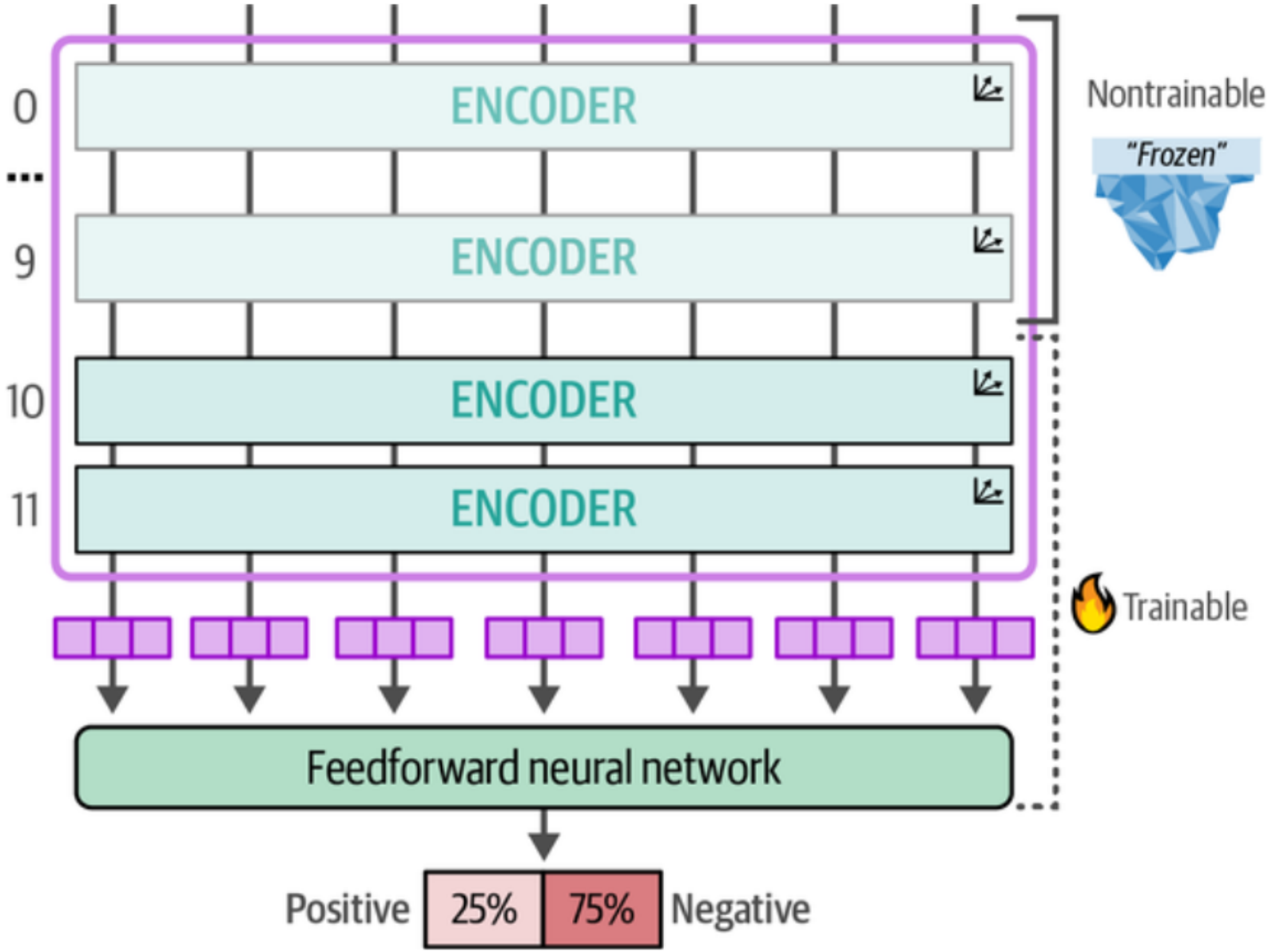
Trade-off:

- Less compute
- Slight performance drop

Empirical observation:

- Training last few blocks often sufficient
- Diminishing returns after several layers

Freezing Layers (cont.)



Few-Shot Classification

Goal:

- Learn from few labeled examples per class

SetFit pipeline:

1. Generate positive and negative sentence pairs
2. Contrastive fine-tuning of embeddings
3. Train classifier on embeddings

Number of positive pairs in class of size n:

$$\frac{n(n-1)}{2}$$

Contrastive Learning in SetFit

Given embeddings u, v:

Positive pairs:

$$\text{maximize similarity}(u, v)$$

Negative pairs:

$$\text{minimize similarity}(u, v)$$

Typical objective:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(u, v^+)/\tau)}{\sum \exp(\text{sim}(u, v)/\tau)}$$

Result:

- Task-aligned embedding space
- Efficient few-shot learning

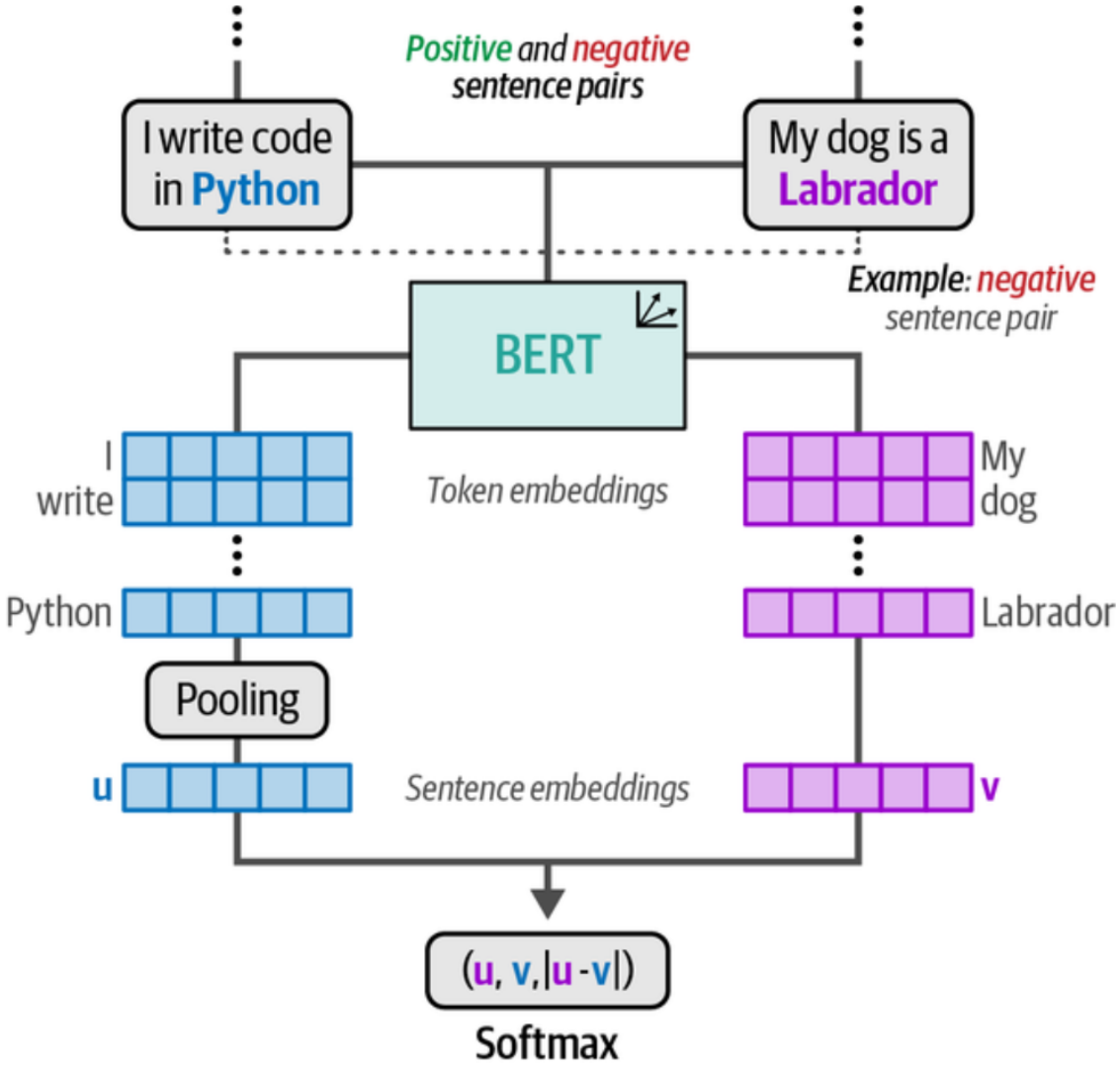
Contrastive Learning in SetFit (cont.)

Text	Class
I write my code in Python	Programming languages
I should practice SQL	Programming languages
My dog is a labrador	Pets
I have a Siamese cat	Pets

Contrastive Learning in SetFit (cont.)

Text 1	Text 2	Pair type
I write my code in Python	I should practice SQL	Positive
My dog is a labrador	I have a Siamese cat	Positive
I write my code in Python	My dog is a labrador	Negative
I have a Siamese cat	I should practice SQL	Negative

Contrastive Learning in SetFit (cont.)



Three-Step Pipeline

Standard:

1. Pretraining
2. Fine-tuning

Extended:

1. Pretraining
2. Continued pretraining on domain data
3. Task fine-tuning

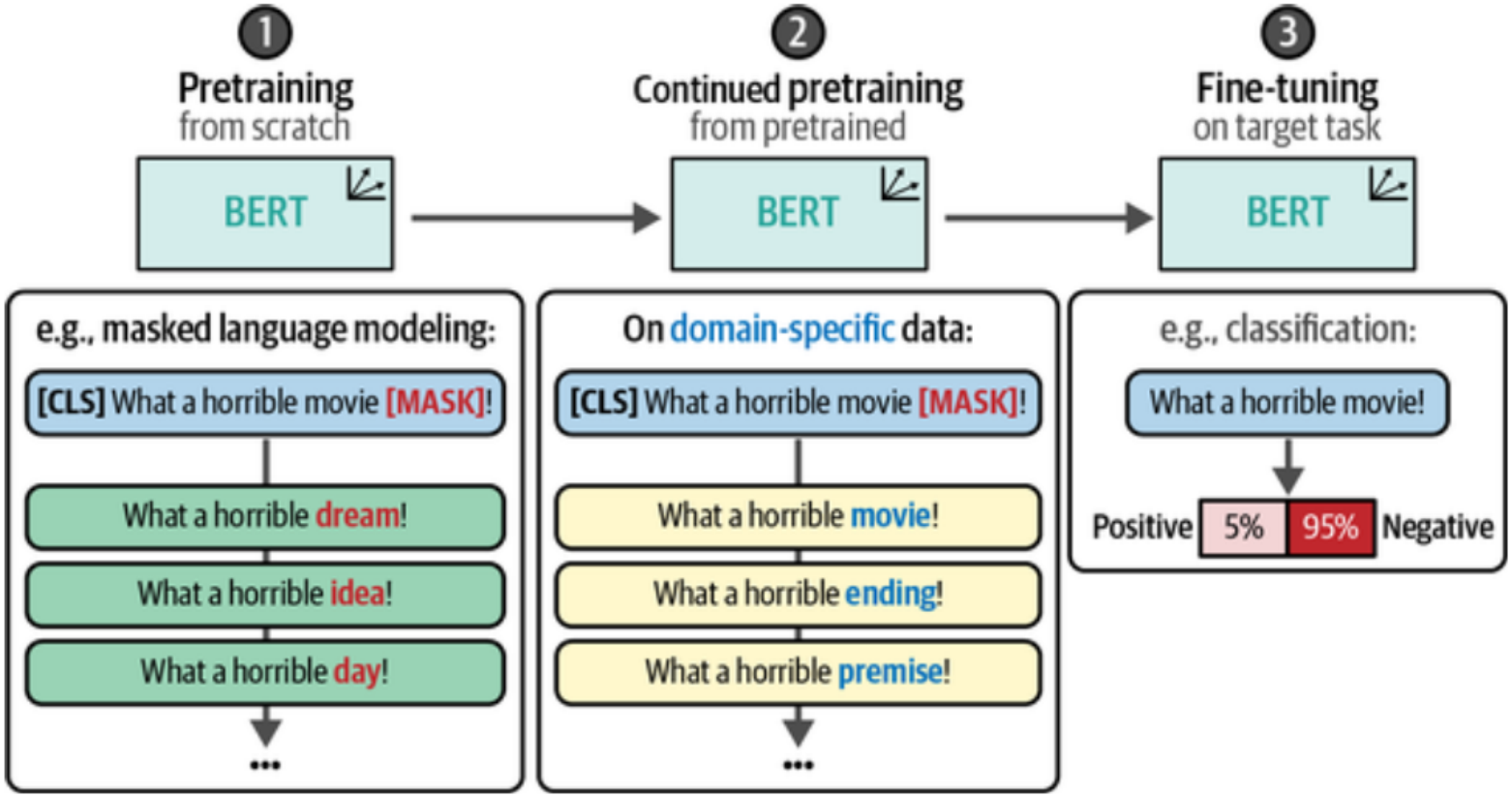
Masked Language Modeling objective:

$$\mathcal{L}_{MLM} = -\sum \log P(x_i \mid x_{\setminus i})$$

Benefit:

- Domain-specific representations
- Improved downstream performance

Continued Pretraining



Three LLM Training Stages

1. Language modeling pretraining
2. Supervised fine-tuning
3. Preference tuning

Language modeling:

$$P(x) = \prod_t P(x_t \mid x_{<t})$$

SFT objective:

$$\mathcal{L}_{SFT} = -\sum \log P(y_t \mid x, y_{<t})$$

Why PEFT?

Full fine-tuning:

- Updates all parameters
- High memory and compute cost
- Large storage footprint

PEFT:

- Freeze base model
- Train small parameter subset
- Efficient and modular

Adapters

Insert small bottleneck modules inside Transformer blocks.

Adapter structure:

$$h' = h + W_{up}\sigma(W_{down}h)$$

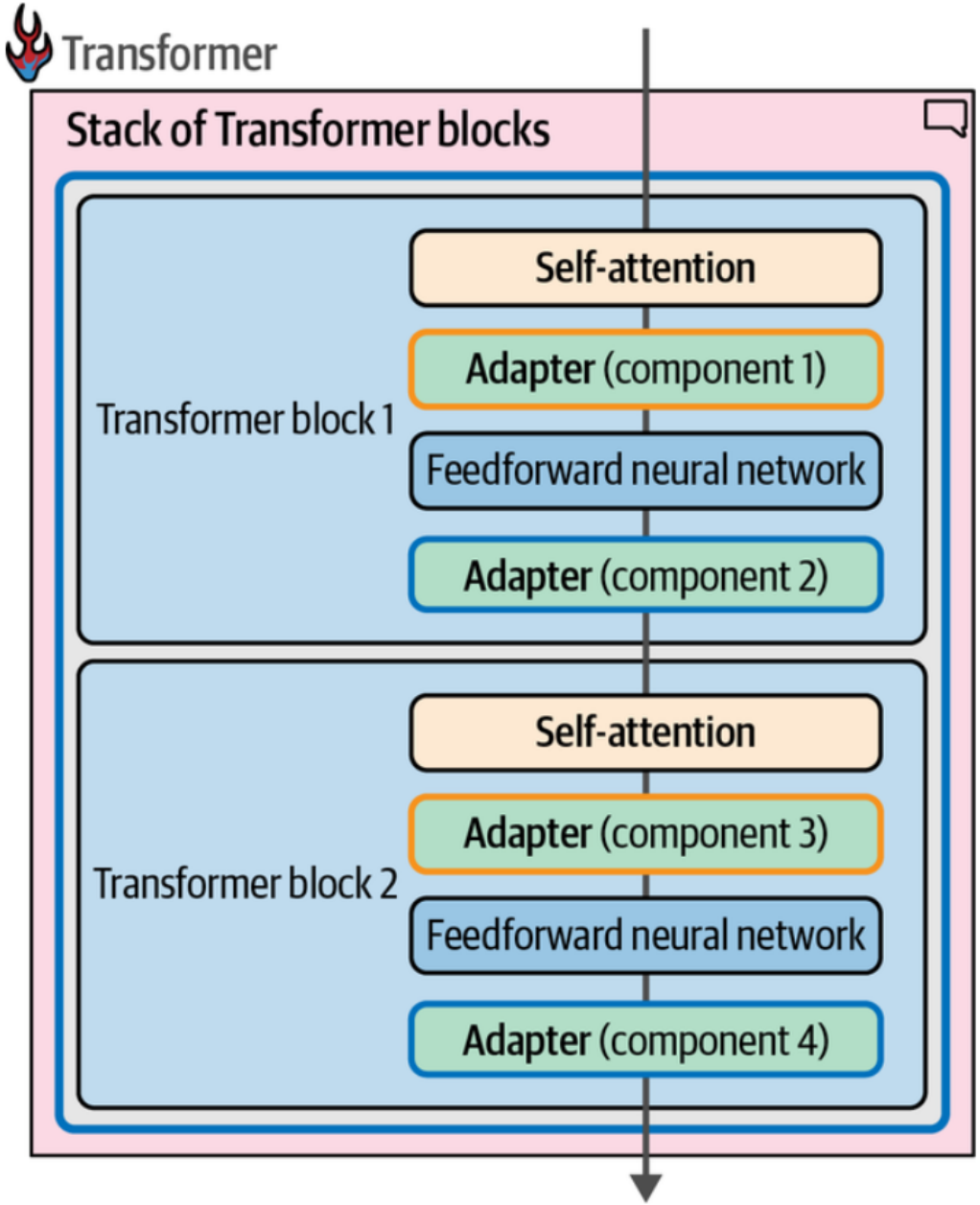
Where:

- $W_{down} \in \mathbb{R}^{d \times r}$
- $W_{up} \in \mathbb{R}^{r \times d}$
- $r \ll d$

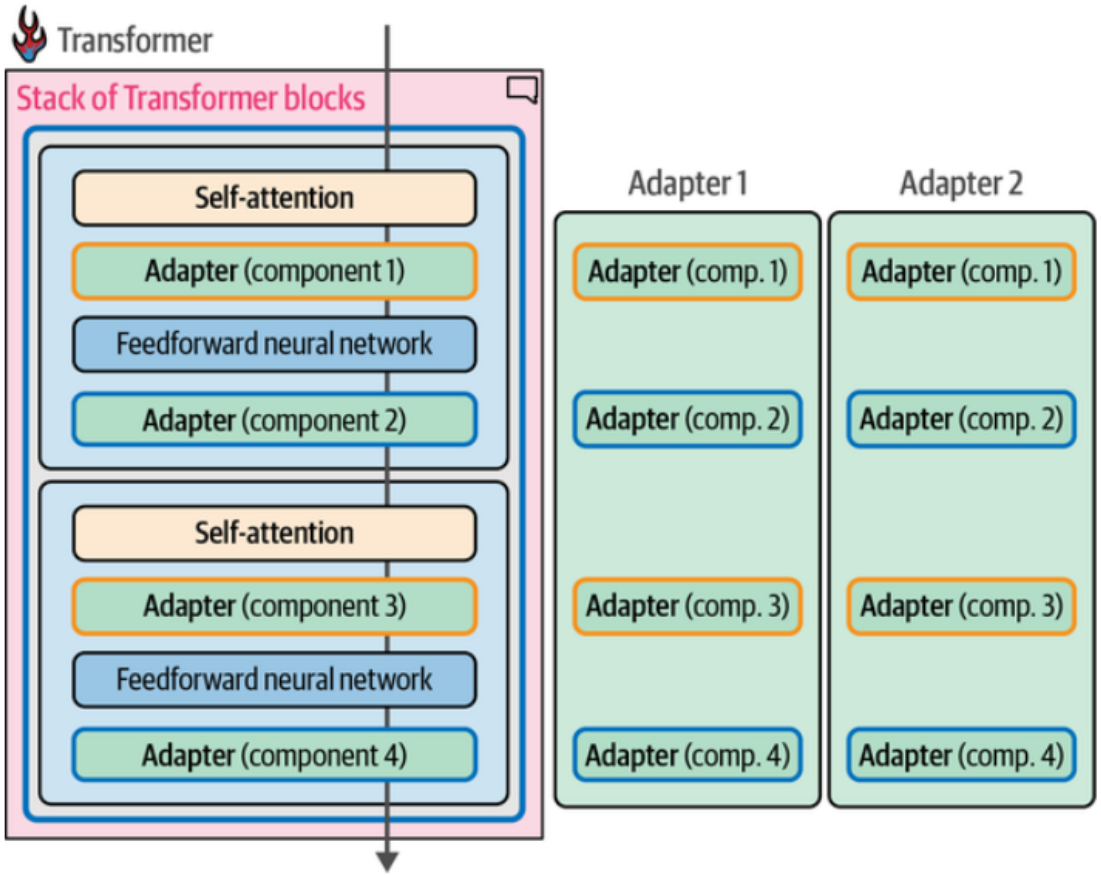
Advantages:

- Few trainable parameters
- Task-specific adapters
- Swap adapters across tasks

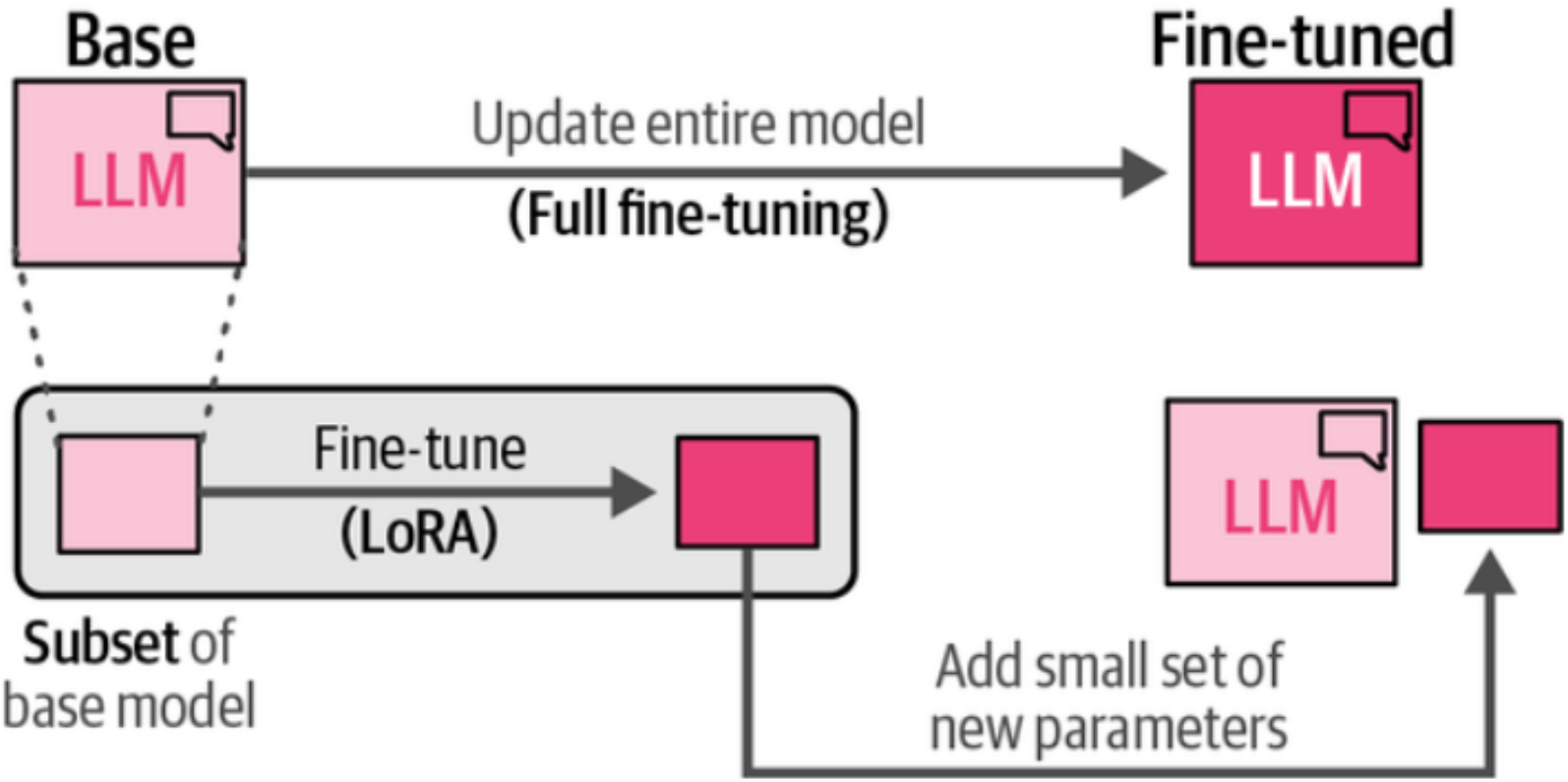
Adapters (cont.)



Adapter Replacement



Low-Rank Adaptation



Low-Rank Adaptation (LoRA)

Original weight:

$$W \in \mathbb{R}^{d \times d}$$

LoRA decomposition:

$$W' = W + BA$$

Where:

- $B \in \mathbb{R}^{d \times r}$
- $A \in \mathbb{R}^{r \times d}$
- $r \ll d$

Trainable parameters:

$$2dr \ll d^2$$

Key idea:

- Intrinsic dimension is low
- Low-rank updates sufficient

How Low Rank is Enforced in LoRA

Key idea: LoRA does not compute a low-rank approximation. It *parameterizes the weight update as low rank from the start.*

Given a pretrained linear layer:

$$W \in \mathbb{R}^{d_{out} \times d_{in}}$$

Full fine-tuning:

$$W \leftarrow W + \Delta W$$

LoRA reparameterizes:

$$W' = W + \Delta W \quad \text{with} \quad \Delta W = BA$$

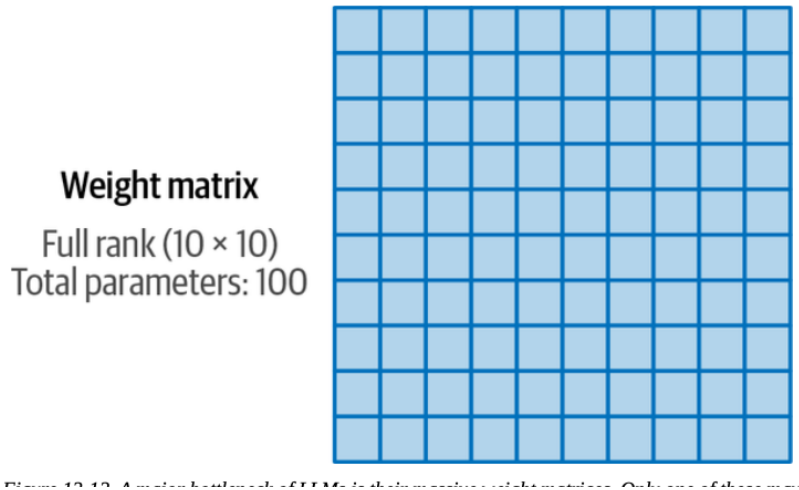
Where:

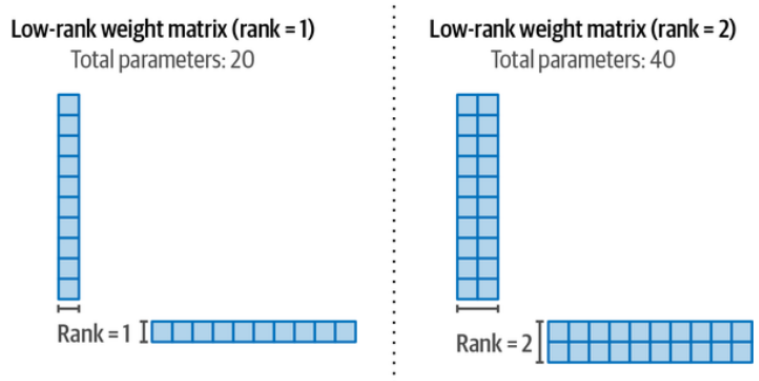
$$B \in \mathbb{R}^{d_{out} \times r}, \quad A \in \mathbb{R}^{r \times d_{in}}, \quad r \ll \min(d_{out}, d_{in})$$

Rank constraint: $\text{rank}(\Delta W) \leq r$

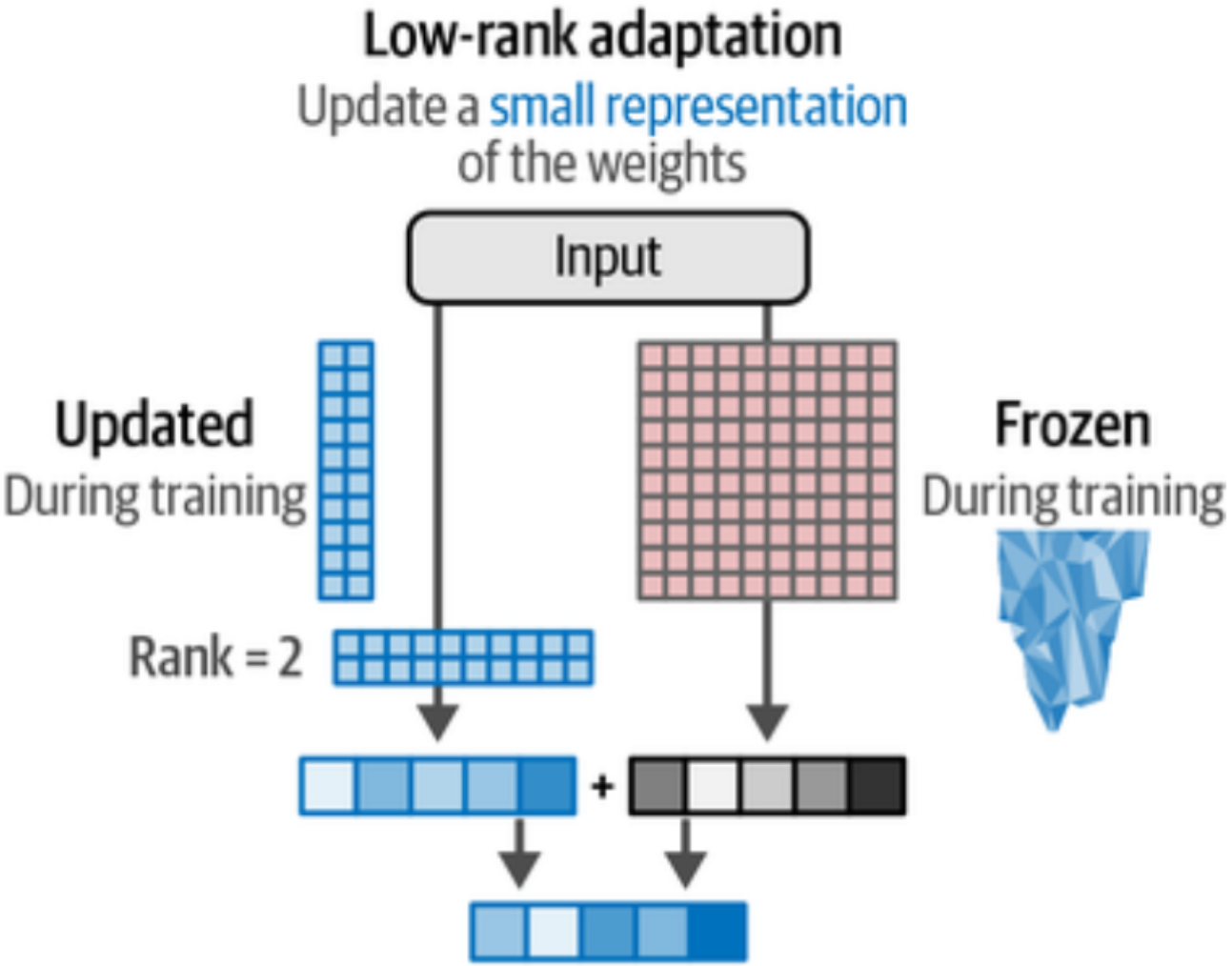
Low rank is enforced structurally, not computed via SVD.

LoRA implementation





Lora Training



LoRA Trade-Offs

Advantages:

- Reduced memory
- Faster training
- Store only LoRA weights

Hyperparameters:

- Rank r
- Scaling factor α
- Target modules

Typical targets:

- Query and value projections

Quantization

Floating point precision:

- FP32
- FP16
- 4-bit

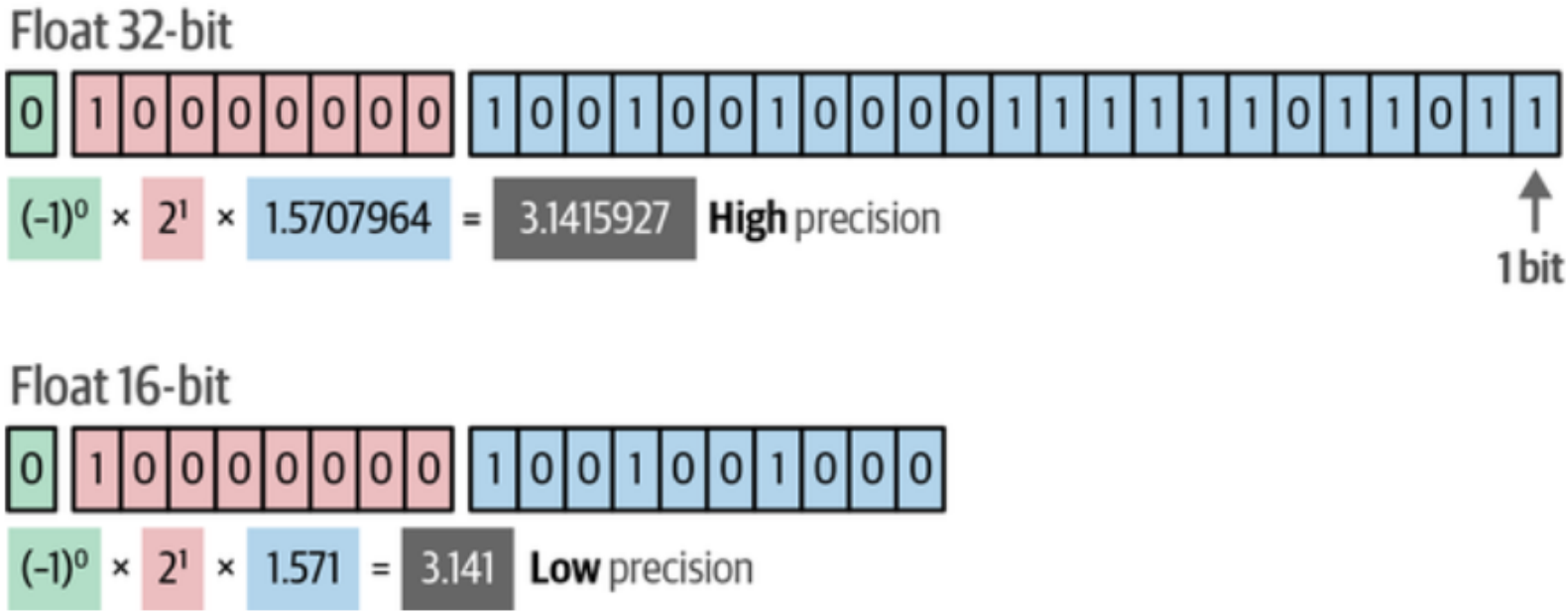
Memory reduction:

$$\text{Memory} \propto \text{bits per weight}$$

Direct quantization issue:

- Loss of precision
- Value collisions

Quantization (cont.)



Blockwise Quantization

Strategy:

- Quantize in blocks
- Normalize per block

Normalized 4-bit (NF4):

- Distribution-aware binning
- Reduces outlier distortion

Combined with LoRA:

- Quantize base model
- Train low-rank adapters

Result:

- Major VRAM reduction
- Minimal performance drop

Prompt Tuning: Core Idea

Goal: Adapt a frozen LLM by learning only input embeddings.

Given pretrained model:

$$f_\theta(x)$$

Freeze θ.

Introduce learnable soft prompt:

$$P = [p_1, \ldots, p_m], \quad p_i \in \mathbb{R}^d$$

Modified input:

$$x' = [P; x]$$

Model output:

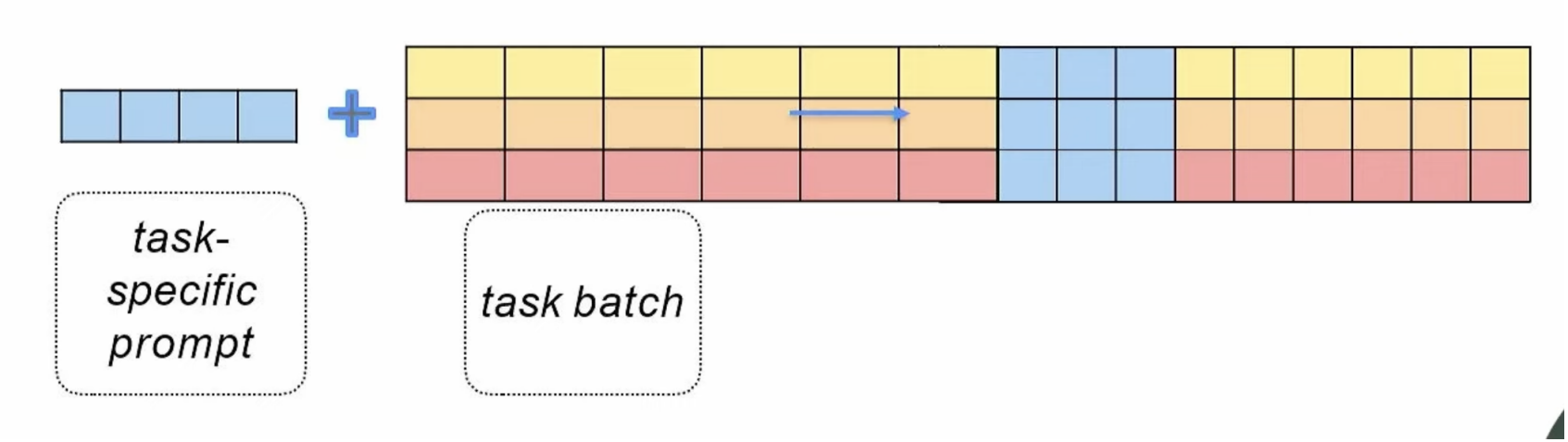
$$y = f_\theta(x')$$

Trainable parameters:

$$|P| = m \cdot d$$

No modification of internal weights. Only prompt embeddings are optimized.

Prompt Tuning



Prompt Tuning: Mechanism and Properties

Optimization Objective

$$\min_{P} \mathbb{E}_{(x,y)}[-\log P_\theta(y \mid [P; x])]$$

Interpretation

- Learned continuous prompt
- Acts as task-specific context
- Steers attention via input embeddings

Capacity

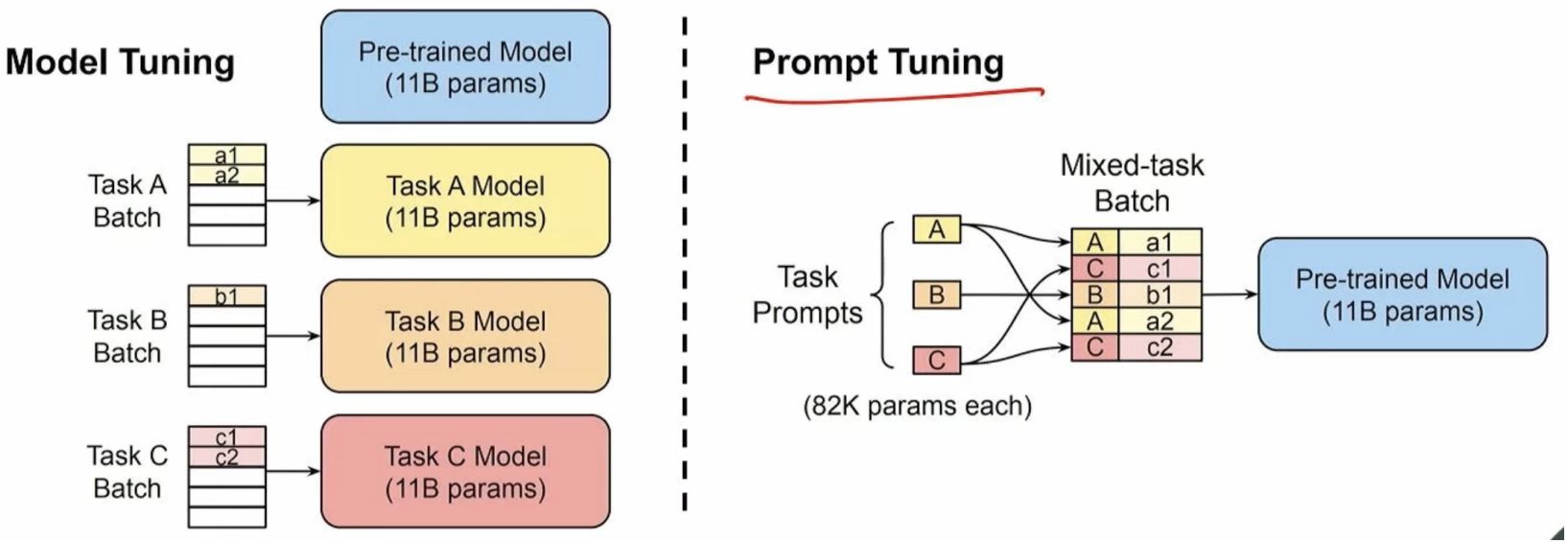
Limited to:

$$\mathcal{O}(md)$$

Typically:

- $m = 10$–100 tokens
- Extremely small parameter footprint

Prompt Tuning Batching



Prefix Tuning: Core Idea

Prompt tuning modifies input only.

Prefix tuning injects trainable vectors into every layer's attention.

For attention layer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Prefix tuning augments:

$$K' = [K_{prefix}; K]$$
$$V' = [V_{prefix}; V]$$

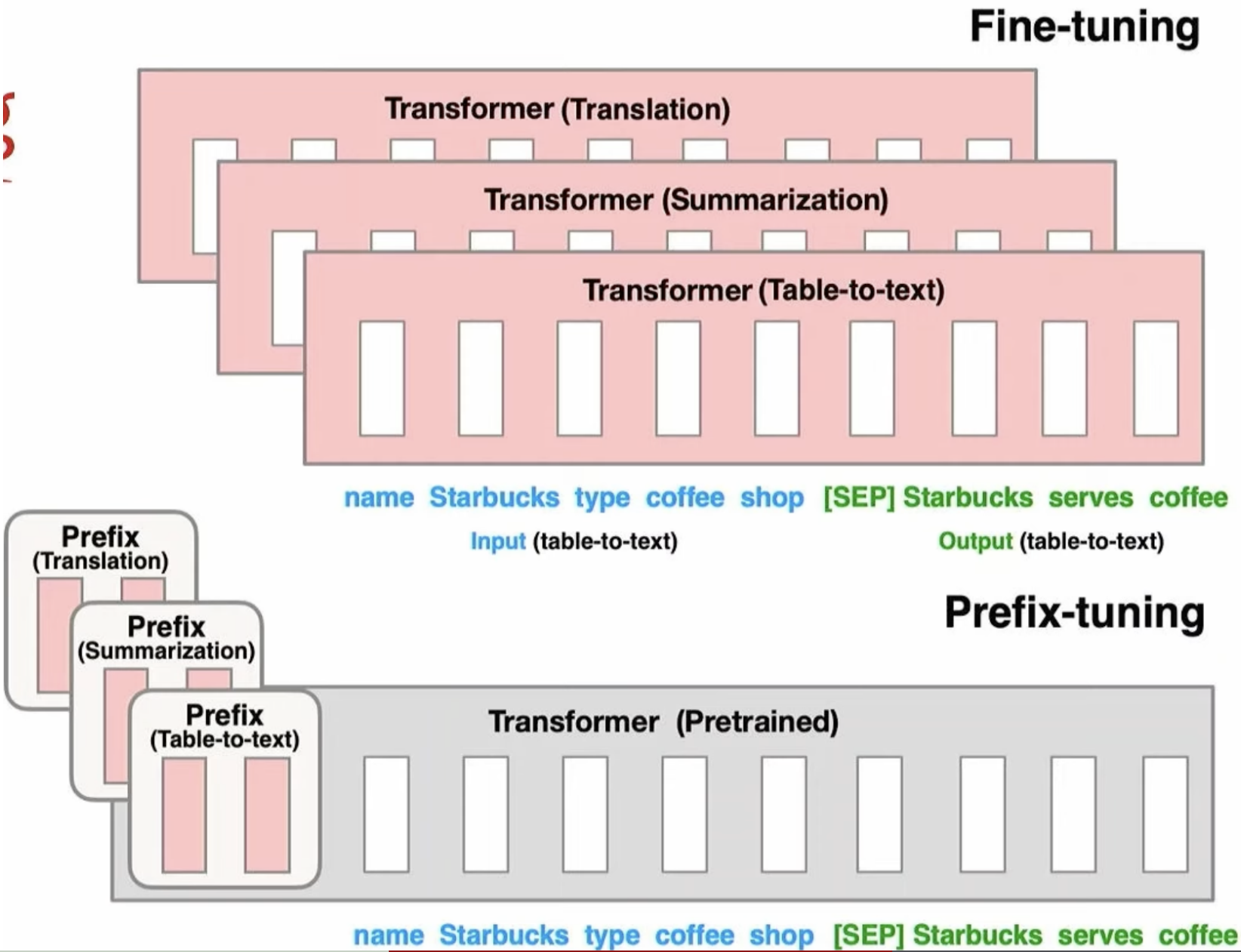
Where:

$$K_{prefix}, V_{prefix}$$

are trainable.

Prefix vectors exist at **each transformer layer**.

Prefix Tuning



Prompt vs Prefix Tuning

Method	Injection Point	Capacity
Prompt Tuning	Input only	$\mathcal{O}(md)$
Prefix Tuning	Every layer (K,V)	$\mathcal{O}(Lmd)$
LoRA	Weight matrices	$\mathcal{O}(2r(d_{in} + d_{out}))$

Trade-offs

Prompt tuning:

- Minimal parameters
- Works better on very large models

Prefix tuning:

- Higher expressive power
- More stable for medium-sized models

Bibliography

Alammar, J. et al. (2024). *Hands-On Large Language Models*. O'Reilly Media.